中国研究生操作系统开源创新大赛

项目创新说明

作 品 名 称：面向智能机器人的 AIOS
参 赛 队 伍：Xenoblade
完 成 日 期：2026 年 10 月

目 录

创新点 1 在 StarryOS 上构建完整的异构硬件视觉链路 **1**
1.1 原因：通用软件路径无法满足闭环时延 1
1.2 过程：补齐驱动接口并建立两级 ION 共享缓冲 1
1.3 效果：图像前端快于 Linux 基线 1
创新点 2 提出面向控制实时性的“最新帧优先”采集机制 **1**
2.1 原因：FIFO 吞吐正确但控制信息过时 1
2.2 过程：异步采集、双缓冲发布与一帧背压 1
2.3 效果：采集与推理重叠且帧龄受控 2
创新点 3 提出面向硬件完成事件的低时延调度方法 **2**
3.1 原因：硬件完成不等于提交线程已经返回 2
3.2 过程：全链路探针、同步直通与短任务优先恢复 2
3.3 效果：软件长尾恢复到真实硬件量级 2
创新点 4 设计任务状态驱动的异构目标感知方法 **2**
4.1 原因：不同任务阶段不需要使用同一种模型 2
4.2 过程：由持球状态选择感知链路 3
4.3 效果：一个小模型完成多目标闭环任务 3
创新点 5 将目标检测结果直接转化为柔性运动控制 **3**
5.1 原因：固定速度会放大视觉闭环延迟 3
5.2 过程：面积反馈、中心死区与多帧确认 3
5.3 效果：从“冲向目标”变为渐进式抵近 3
创新点 6 以“每次操作的固定成本”为口径，把通用子系统与 Linux 逐项对齐 **4**
6.1 原因：差距不在带宽，而在每次操作要付的那一笔 4
6.2 过程：先归因、再改动、每步留回退 4
6.3 效果：一个数量级的差距收到个位数倍数 4
创新点 7 把“存在但未被使用”的协处理器做成第二个可编程执行体 **4**
7.1 原因：单核平台的实时缺口与单点失效 4
7.2 过程：先打通核间通道，再交出一整条链路 5
7.3 效果：抖动与故障域都变成可承诺的量 5
小 结 **6**

创新点 1　在 StarryOS 上构建完整的异构硬件视觉链路

1.1　原因：通用软件路径无法满足闭环时延

项目初期的摄像头采集、JPEG 解码、颜色转换、缩放和张量准备主要依赖 CPU 或分散的设备缓冲区完成。对于单核 C906，这不仅产生逐像素计算，还使 640×480 的 YUV 图像和 640×640 的 RGB 图像在用户缓冲区、设备缓冲区与模型张量之间反复复制。更关键的是，摄像头 JPEG 实际采用 4:2:2 色度采样，JPU 输出为 YUV422P，而原有 VPSS 接口主要支持 NV12，链路中存在不能通过简单格式标记解决的硬件接口断点。

1.2　过程：补齐驱动接口并建立两级 ION 共享缓冲

本项目补齐 UVC、JPU、VPSS、ION 与 TPU 五类驱动能力，形成端到端硬件链路：

$$\text{MJPEG} \rightarrow \text{JPU} \rightarrow \text{YUV422P} \rightarrow \text{VPSS} \rightarrow \text{RGB Planar} \rightarrow \text{TPU}$$

驱动新增 YUV422P 三平面 VPSS 接口，显式传递 Y、Cb、Cr 物理地址、偏移和步长，由一次离线任务完成 BT.601 颜色转换、letterbox 和 RGB planar 排布。source ION 同时作为 JPU 输出与 VPSS 输入，destination ION 同时作为 VPSS 输出与 TPU 输入；内核从文件描述符解析低 4 GiB 物理地址，并校验 DMA32、平面范围和所有权状态。由此，614,400 字节 YUV422P 和 1,228,800 字节 RGB planar 不再由 CPU 整图搬运。

1.3　效果：图像前端快于 Linux 基线

正式测试中，JPU 和 VPSS 平均时延分别为 1.645 ms 和 1.643 ms，“摄像头等待+JPU+VPSS”合计 8.776 ms，比 Linux 基线 12.949 ms 低 4.173 ms。StarryOS 单帧视觉总时延为 53.024 ms，在相同硬件、相同 640×640 模型下低于 Linux 基线的 54.23 ms，并保持 93.13% 的平均检测置信度。

创新点 2　提出面向控制实时性的“最新帧优先”采集机制

2.1　原因：FIFO 吞吐正确但控制信息过时

先进先出队列强调处理每一幅图像；当摄像头帧率高于视觉处理速度时，未处理帧会持续积压。即使系统吞吐率没有下降，机器人依据的目标位置也已经落后于真实环境，容易造成转向过冲和抓取点漂移。移动机器人控制更关心图像帧龄；完整处理所有历史帧无法保证决策所依据的环境状态足够新，因此采集策略必须从“无丢帧”转向“最新状态优先”。

2.2　过程：异步采集、双缓冲发布与一帧背压

内核实现 `AsyncCapture` 后台任务和 `LatestFrameStore` 双缓冲。生产者只向非活动缓冲区组装 USB/UVC 数据，在 SOI、EOI 和长度校验全部通过后才原子切换发布索引并递增帧序号；消费者携带 `last_sequence` 请求新帧，落后时直接读取最新完整快照，不必清空历史队列。为避免采集任务持续占用单核 CPU，生产者仅在上一帧被 `mark_consumed()` 确认后预取下一帧，形成容量为一帧的有界生产者—消费者协议。损坏帧则在进入 JPU 前被识别、丢弃并有限重试。

2.3　效果：采集与推理重叠且帧龄受控

该机制使 UVC 采集能够在前台处理当前帧时准备下一帧，同时保证消费者不会读取半帧、重复帧或积压帧。正式测试的摄像头等待平均为 5.489 ms；该数值包含等待一个新序号的真实语义，与从已有 FIFO 队列中立即 取出旧帧的计时含义不同。系统在保持有界资源占用的同时，为后续运动控制提供更接近当前环境的图像。

创新点 3　提出面向硬件完成事件的低时延调度方法

3.1　原因：硬件完成不等于提交线程已经返回

早期日志中，VPSS 和 TDMA 均出现 40 ms 以上的墙钟时间，但分段测量表明 VPSS 硬件执行为 1.480 ms，异常 TDMA 样本的真实搬运为 0.838 ms。单核 StarryOS 采用 RR 调度，等待线程在 IRQ 到达后仍可能排在运行队列尾部；TPU 旧路径还需要“提交线程—worker—提交线程”两次任务所有权转移。若只在 ioctl 入口和出口计时，这些排队与恢复成本 会被误判为硬件性能不足。

3.2　过程：全链路探针、同步直通与短任务优先恢复

项目在用户态 API 边界和内核设备生命周期中建立分段探针，将任务拆为提交、启动、`fire_to_irq`、`irq_to_resume` 和返回用户态等区间。VPSS 使用 3 ms 有界完成观察规避正常任务的长时间片睡眠；TPU 改为 提交线程直接执行 `prepare → fire → wait → finish`，硬件运行期间仍睡眠等待 IRQ，但不再转交中间 worker。TDMA-only 任务则使用 IRQ 队首唤醒，并把兜底轮询从 100 微秒调整为 2 ms，修复短周期轮询提前移除等待者、 导致真实 IRQ 无法优先唤醒的竞态。

3.3　效果：软件长尾恢复到真实硬件量级

代表性 A/B 测试中，VPSS 墙钟由 44.646 ms 降至 1.637 ms；TPU 的 `submit_to_run` 由 22.873 ms 降至 0.002 ms，`done_to_user` 由 2.958 ms 降至 0.116 ms；TDMA-only 内核任务由 45.253 ms 降至 0.978 ms。正式测试中的 输入 TDMA 为 1.114 ms，TPU 推理为 41.309 ms，其中 compute 内核时间的 97% 以上已经是真实硬件命令流。 这表明优化后 TPU 驱动的软件路径已达到与官方 Linux 驱动相同量级，剩余主要瓶颈转移到模型本身。

创新点 4　设计任务状态驱动的异构目标感知方法

4.1　原因：不同任务阶段不需要使用同一种模型

机器人需要识别网球和红色接球容器，但测试环境中红色容器具有稳定、显著的颜色特征。若重新训练并 部署多类别模型，会增加数据标注、模型输出和推理负担；若任何时候都响应红色区域，机器人又可能在 尚未抓到球时错误驶向容器。因此，感知算法必须同时考虑目标特征与任务状态。

4.2　过程：由持球状态选择感知链路

未持球时，机器人使用 YOLOv8 与 TPU 检测多个网球，经过阈值过滤和 NMS 后选择面积最大的目标；抓球并 确认 `holding_ball=true` 后，状态机切换到 YUV422P 颜色检测，直接在 JPU 输出上统计红色像素、质心和外接框， 不再执行该阶段无用的 VPSS 与 TPU：

$$\text{未持球} \Rightarrow \text{YOLO 网球检测}$$

$$\text{已持球} \Rightarrow \text{红色容器检测}$$

红色容器只有在持球状态下才是有效目标；其外接框达到画面宽度 100%、高度 98% 且连续三帧确认后，系统 才停车放球。放球后先直线后退恢复视野，再以低速原地旋转搜索，并设置 20 帧保护窗口避免重新追逐刚放下 的网球。

4.3　效果：一个小模型完成多目标闭环任务

该方法没有扩大 YOLO 模型，却同时具备多网球实例选择和红色容器识别能力。红色搜索阶段按需删除 VPSS 和 TPU 任务，避免无效计算；状态门控又阻止空夹时误追红车。机器人由此能够完成“选取近球—抓取—寻找 容器—放球—退出遮挡—重新搜索”的连续循环，并通过 `--max-deposits N` 指定任务球数。

创新点 5　将目标检测结果直接转化为柔性运动控制

5.1　原因：固定速度会放大视觉闭环延迟

摄像头采集、推理、串口下发和底盘制动均存在有限延迟。采用固定高速接近时，机器人在看到近球后仍会 带着较大动量前冲，造成撞球；采用简单左右纠偏又容易在目标中心附近反复摇摆。因此，检测结果必须被 转换为连续的速度反馈，并与离散的转向状态协同使用。

5.2　过程：面积反馈、中心死区与多帧确认

系统使用检测框水平中心估计方向，用检测框面积比例 a 近似目标距离，并设计二次减速函数：

$$v(a) = v_{\text{chase}} \left(1 - \frac{a}{a_{\text{grab}}}\right)^2$$

实现中将最大追踪速度由 45 降至 27，最低有效速度设为 6，抓取面积阈值设为 0.55。目标偏离夹爪中心时， 底盘使用中心死区和短时差速脉冲纠偏；进入近球区域后扩大中心容差，只有位置与面积连续两帧满足条件才 停车抓取。机械臂仍沿用经过验证的抓球序列，放球阶段则使用独立的空夹角度和撤离动作。

5.3　效果：从“冲向目标”变为渐进式抵近

二次曲线使机器人在远距离保持必要速度，在中距离主动减速，在近距离以最低有效速度缓行，达到阈值后 稳定停车。相对于固定或分段线性策略， 车辆更少出现近距离过冲、撞球和左右摇摆；结合最大检测框优先、抓取多帧确认和放球后脱困，控制程序能够连续处理多颗网球，而不需要人工在每次抓取后复位状态。

创新点 6　以“每次操作的固定成本”为口径，把通用子系统与 Linux 逐项对齐

6.1　原因：差距不在带宽，而在每次操作要付的那一笔

在同一块板、同一份符合 Linux ABI 的可执行文件下，两套系统的差别集中在“固定开销”上。优化前 九个系统调用呈同一条水平线——处理体只从当前线程结构里读一个字段的 `getpid` 也要 10 560 ns， 而同板 Linux 只要 422.5 ns；与此同时，`memcpy` 与顺序读的带宽只差几个百分点。也就是说， 差距与数据量无关，来自每次系统调用、每次进程创建、每次缺页、每次文件操作都要多付的那一笔。

6.2　过程：先归因、再改动、每步留回退

三种手法贯穿始终。分段计账把一条路径切成若干段累计耗时，经 `/proc/fault_attrib` 取出，按“工作负载前后的快照之差”折算成每次调用的平均值；标度实验固定其它条件、只改一个自变量，看哪一段成本随之变化；受控 A/B 在同一次上电、交替顺序、各测两遍取均值，用来抵消本板上“启动后第一次大分配更快”这类漂移。

据此落地的改动按机制族归并：返回用户态的指令缓存刷新从“每次都刷”改成只在映像发生变化时刷 一次；COW 页索引去掉每次插入的全量重建，内核导出的 `memcpy`/`memmove`/`memset`/`memcmp` 从逐字节循环改成字长批量；`execve` 只物化带写权限的段并按 run 批量 `pin`；顺序缺页按 64 KiB 窗口批量物化，窗内帧整块分配后切成子租约，索引按 run 批量登记；写回按连续页合并 下发、页缓存上限由 512 页提到 4096 页、`close` 按 Linux 语义处理访问时间。凡动到对外语义 的改动（指令缓存、TLB、访问时间）都保留运行期回退开关（`/proc/icache_flush`、`/proc/fault_around`）。

6.3　效果：一个数量级的差距收到个位数倍数

空系统调用由 25.0× 收到 3.9×，4 KiB 管道往返由 10.4× 收到 2.8×，`fork`+`execve`+`waitpid` 由 18.1× 收到 9.3×，ext4 `open`+`read`+`close` 由 246× 收到 10.8×；64 MiB 匿名首触的每页 成本由 1 302 123 ns 降到 44 120 ns，与 Linux 的差距由 165× 收到 5.6×，并且不再随区域大小 增长。带宽型与总量型指标本来就不落后：8 MiB 页缓存顺序写由 2.41 MiB/s 提到 42.43 MiB/s （同板 Linux 为 34 MiB/s），8 MiB 顺序写 + `fsync` 由 7 961 ms 降到 931 ms。

同一套方法也被用来否掉不值得做的方向。六个假设经实测排除——缺页慢是因为刷 TLB、页缓存 容量不够、缓存注册表裁剪是 $O(n^2)$、`fork` 慢是因为首次入队抢占、无线链路失效是协议缺陷、 页索引插入可以对整段简单合入（后者第一版实现让板子在启动时硬挂）。把这些负结果与改动一起 写进报告，使“没做什么”与“做了什么”同样可核对。

创新点 7　把“存在但未被使用”的协处理器做成第二个可编程执行体

7.1　原因：单核平台的实时缺口与单点失效

本板只有一颗通用核参与调度，片内那颗 RISC-V C906@700 MHz 协处理器（C906L）被厂商固件 独占。于是摄像头取流、JPEG 解码、TPU 推理、跟踪决策与串口下发全部排在同一个时间

创新方向	核心实现	验证效果
异构视觉链路	YUV422P 原生 VPSS 接口与两级 ION 共享缓冲	图像前端 8.776 ms，总时延 53.024 ms
最新帧采集	异步 UVC、双缓冲、序号同步与一帧背压	采集与推理重叠，帧龄和资源占用有界
完成事件调度	同步直通、TDMA 分类、IRQ 前插与轮询竞态修复	VPSS 1.637 ms，TDMA 内核任务 0.978 ms
状态驱动感知	YOLO 网球检测与 YUV 红色检测按持球状态切换	单一小模型完成抓球、放球与再次搜索
柔性运动控制	二次减速、中心死区、多帧确认和遮挡脱困	降低撞球、过冲与近距离左右摇摆
内核固定成本	分段归因、八族机制改动与运行期回退开关	空系统调用 25.0×→3.9×，64 MiB 首触 165×→5.6×
实时小核启用	核间通道、底盘链路接管、心跳看门狗与堵转保护	控制周期由负载相关变为 0 抖动，主核失效后能自主停车

表 1: 项目创新的实现落点与验证效果

片上：空载时底盘控制周期 p99 相对 100 ms 只偏 0.21 ms，一旦有 3 个纯计算负载把核压满就恶化到 1.54 ms（约 8 倍），同一时刻串口往返由 120–390 µs 涨到 3 670 µs。比抖动更严重的是单点 失效：底盘板执行“最后一条速度指令一直有效”，主核崩溃、卡死或重启会连带带走“停车”这个 动作。

7.2　过程：先打通核间通道，再交出一整条链路

通道被实现为四层：驱动层接管 `ap_mailbox` 的 8 个槽位与中断位；锁与唤醒层按“硬中断 → 服务线程 → 等待队列”三段式实现，中断上下文只用 IRQ-save 自旋锁与固定容量队列；共享内存层 提供 1 MiB 非缓存保留区与 v3 布局（头部按写入方拆 cache line、双 64 槽环 + CRC32）；接口层 把能力暴露成 `/dev/cvi-rtos-cmdqu`，支持指定应答命令号的 `EXCHANGE` 与 `poll/epoll`。

随后让这颗核接管底盘串口，分三步各自设判据：独立握手（拔线冻结、插回恢复）、协议自检 （全程零速度指令）、周期服务与心跳看门狗（100 ms 周期、500 ms 心跳窗口），并加一层不依赖 新增硬件的堵转与碰撞保护；机械臂舵机由另一路 UART 直接驱动。核的定位随之明确——只接实时与 兜底职责，不承担推理与图像预处理。

7.3　效果：抖动与故障域都变成可承诺的量

同机、同负载、同链路的 A/B 给出结论：大核直控在 3 个计算负载下 p99 恶化到 1.54 ms、最坏 1.75 ms，小核接管在全部负载下 min = mean = max，样本 100% 落在期望的 tick 刻度；大核停喂 心跳约 500 ms 后小核自主停车，小核被停掉后大核侧的只读判据在 2 s 内确认并抢回链路补发停车 序列。收益不在算力，而在确定性、系统无关与故障域隔离——这与内核那条线互补：后者把每次 操作的成本降到接近 Linux，前者把不能靠降成本解决的时序问题交给另一颗核。

小结

本项目的核心创新，是面向移动机器人实时闭环做跨层协同设计。应用层以“最新帧优先”的采集与面向 硬件完成事件的调度，把图像链路与运动控制串成闭环；通用子系统层以“先归因、再改动、每步留回退” 的方法，把每次操作的固定成本从十几倍到上百倍的差距收到个位数倍数；板级则把厂商固件里闲置的 协处理器做成可管理的第二个执行体, 用它与上板工作流同时补上确定性与可复现性。系统既解决了 YUV422P 格式断点、ION 共享缓冲、JPU 边界预取和 TPU 运行时兼容等数据面问题，也解决了 worker 交接、IRQ 恢复、轮询竞态与 COW 页索引等控制面与内核路径问题。最终，单帧视觉时延为 53.024 ms， 比同硬件 Linux 基线低 2.22%；内核侧空系统调用的差距收到 3.9×、64 MiB 匿名首触收到 5.6×； 底盘控制周期在全部负载下 100% 落在期望 tick 刻度，主核失效时车能在心跳窗口内自主停车。